6.2 Dependency Injection

Part of Services and Dependency Injection Module - Duration: 10-12 hours

1. What is Dependency Injection?

1.1 The Problem: Creating Dependencies Manually

Before understanding Dependency Injection (DI), let's see the problem it solves. Consider a component that needs to use a service:

```
// Without Dependency Injection (Bad Approach)
import { Component } from '@angular/core';
import { ProductService } from './product.service';

@Component({
  selector: 'app-product-list',
  template: `<div>Product List</div>`
})
export class ProductListComponent {

  productService: ProductService;

  constructor() {
    // Component creates its own service instance
    this.productService = new ProductService();
  }

  getProducts() {
    return this.productService.getAllProducts();
  }
}
```

Output:

Problems with this approach:

- Component is tightly coupled to ProductService
- Hard to test (can't replace with mock service)

- Must manually create all service dependencies
- No sharing of service instances between components
- Component must know how to construct the service

1.2 The Solution: Dependency Injection

Dependency Injection is a design pattern where a class receives its dependencies from external sources rather than creating them itself. Angular's DI system automatically provides the required dependencies when a class is instantiated.

```
// With Dependency Injection (Good Approach)
import { Component } from '@angular/core';
import { ProductService } from './product.service';

@Component({
  selector: 'app-product-list',
  template: `<div>Product List</div>`
})
export class ProductListComponent {

  // Angular automatically injects the service
  constructor(private productService: ProductService) { }

  getProducts() {
    return this.productService.getAllProducts();
  }
}
```

Output:

Benefits of Dependency Injection:

- Component doesn't create the service - Angular provides it
- Easy to test with mock services
- Services can be shared across components
- Loose coupling between components and services
- Angular manages service lifecycle automatically

1.2.1 Simple Examples: JavaScript → TypeScript

Small concrete examples that show the same DI idea outside and inside TypeScript/Angular.

JavaScript (plain) — constructor injection

Pass dependencies into the consumer so they can be swapped or mocked.

```
// Logger.js
class Logger {
  log(msg) { console.log('[LOG]', msg); }
}

// App.js (consumer)
class App {
  constructor(logger) { this.logger = logger; }
  run() { this.logger.log('App started'); }
}

// composition
const logger = new Logger();
const app = new App(logger);
app.run();

// For tests you can swap in a fake logger:
class FakeLogger { constructor(){ this.messages = [] } log(m){ this.messages.push(m) } }
const fake = new FakeLogger();
const testApp = new App(fake);
// testApp.run(); assert(fake.messages.includes('App started'))
```

TypeScript — constructor injection + interface

Use an interface for the dependency, then inject a concrete implementation.

```
// ILogger.ts
export interface ILogger { log(msg: string): void }

// ConsoleLogger.ts
import { ILogger } from './ILogger';
export class ConsoleLogger implements ILogger {
  log(msg: string){ console.log('[LOG]', msg); }
}
```

```
// App.ts
import { ILogger } from './ILogger';
export class App {
  constructor(private logger: ILogger) {}
  run(){ this.logger.log('App started (TS)'); }
}

// usage
const logger = new ConsoleLogger();
const app = new App(logger);
app.run();
```

> **Important:**
>
> Why this helps: the consumer (`App`) is decoupled from concrete implementations. You can replace loggers for testing or swap behaviour without changing `App`.

1.3 Inversion of Control (IoC) Principle

Dependency Injection is an implementation of the Inversion of Control principle. Instead of your code controlling when and how dependencies are created, the framework (Angular) takes control.

```
// Traditional Control Flow
Component creates Service
    ↓
Component controls Service lifecycle
    ↓
Component destroys Service

// Inverted Control (DI)
Framework creates Service
    ↓
Framework injects Service into Component
    ↓
Framework manages Service lifecycle
    ↓
Framework controls when Service is destroyed
```

> **Important:**
>
> **Inversion of Control means:** You don't call the framework; the framework calls you. You declare what you need (dependencies), and Angular provides them.

1.4 Real-World Analogy

Think of DI like ordering food at a restaurant:

Without DI (Cooking at Home)	With DI (Restaurant)
You buy ingredients	You order from menu
You prepare the food	Kitchen prepares the food
You clean the dishes	Restaurant handles cleanup
You control everything	Restaurant provides everything

Just like you don't need to know how the restaurant kitchen works, your component doesn't need to know how to create and manage services - Angular's DI system handles it all.

2. How Angular DI Works

2.1 The Three Key Players

Angular's Dependency Injection system involves three main concepts:

2.1.1 Injector

The injector is responsible for creating service instances and injecting them into components or other services. Angular creates a hierarchy of injectors throughout the application.

```
// Conceptual representation
Injector
  ├── Creates service instances
```

```
   ├── Maintains service instances (singletons)
   ├── Injects dependencies when requested
   └── Manages service lifecycle
```

2.1.2 Provider

A provider tells the injector how to create a service. It's like a recipe that describes how to instantiate the service.

```
@Injectable({
  providedIn: 'root'  // ← This is provider configuration
})
export class DataService { }
```

2.1.3 Dependency

The dependency is what a component or service needs. It's declared in the constructor.

```
constructor(private dataService: DataService) { }
//          ↑ This declares a dependency
```

2.2 Injection Process Step-by-Step

Let's trace what happens when Angular injects a service:

```
// Step-by-step DI process

// 1. Component declares dependency
@Component({
  selector: 'app-user-profile'
})
export class UserProfileComponent {
  constructor(private userService: UserService) { }
}

// 2. Angular checks if UserService exists
// 3. If not created, Angular looks for provider
```

```
// 4. Angular creates UserService instance
// 5. Angular injects instance into component
// 6. Component can now use userService
```

Output:

Injection Flow:

Request → Check Injector → Find Provider → Create/Retrieve Instance → Inject → Use

2.3 Practical Example: Full DI Workflow

Let's see a complete example with service creation, injection, and usage:

```
// 1. Create the service
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'  // Provider configuration
})
export class CounterService {

  private count: number = 0;

  increment(): void {
    this.count++;
  }

  decrement(): void {
    this.count--;
  }

  getCount(): number {
    return this.count;
  }

  reset(): void {
    this.count = 0;
  }
}
```

```
// 2. Inject into first component
import { Component } from '@angular/core';
import { CounterService } from './counter.service';

@Component({
  selector: 'app-counter-display',
  template: `
    <div>
      <h3>Counter Display</h3>
      <p>Count: {{ getCount() }}</p>
      <button (click)="increment()">+</button>
      <button (click)="decrement()">-</button>
    </div>
  `
})
export class CounterDisplayComponent {

  constructor(private counterService: CounterService) { }

  increment(): void {
    this.counterService.increment();
  }

  decrement(): void {
    this.counterService.decrement();
  }

  getCount(): number {
    return this.counterService.getCount();
  }
}
```

```
// 3. Inject into second component
import { Component } from '@angular/core';
import { CounterService } from './counter.service';

@Component({
  selector: 'app-counter-controls',
  template: `
    <div>
      <h3>Counter Controls</h3>
      <p>Current: {{ getCount() }}</p>
      <button (click)="reset()">Reset</button>
    </div>
```

```
  `
})
export class CounterControlsComponent {

  constructor(private counterService: CounterService) { }

  reset(): void {
    this.counterService.reset();
  }

  getCount(): number {
    return this.counterService.getCount();
  }
}
```

Output:

Result: Both components share the SAME CounterService instance. When CounterDisplayComponent increments the counter, CounterControlsComponent sees the same updated value. This is because Angular provides a single instance (singleton) of the service.

3. Providers: Configuring Services

3.1 Understanding Providers

Providers tell Angular how and where to create service instances. There are multiple ways to configure providers, each with different scopes and behaviors.

3.2 Root-Level Provider (Application-Wide)

Using `providedIn: 'root'` makes the service available throughout the entire application as a singleton.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'  // Available everywhere
})
```

```
export class GlobalDataService {
  data: string = 'Global data';
}
```

Output:

Characteristics:

- **Scope:** Entire application
- **Instances:** One singleton instance
- **Shared:** All components use same instance
- **Tree-shakable:** Removed if unused
- **Recommended:** Yes, for most services

3.3 Module-Level Provider

Services can be provided in a module's `providers` array, making them available to all components declared in that module.

```
// auth.service.ts
import { Injectable } from '@angular/core';

@Injectable()  // No providedIn
export class AuthService {
  isAuthenticated: boolean = false;

  login(username: string, password: string): boolean {
    // Simple authentication logic
    if (username === 'admin' && password === 'admin123') {
      this.isAuthenticated = true;
      return true;
    }
    return false;
  }

  logout(): void {
    this.isAuthenticated = false;
  }

  isLoggedIn(): boolean {
```

```
    return this.isAuthenticated;
  }
}
```

```
// app.module.ts
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { AuthService } from './auth.service';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  providers: [AuthService],  // Provided at module level
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Output:

Characteristics:

- **Scope:** All components in the module
- **Instances:** One instance per module
- **Shared:** Within module components only
- **Tree-shakable:** No
- **When to use:** Legacy code or specific module-scoped services

3.4 Component-Level Provider

Services can be provided at the component level, creating a new instance for that component and its children.

```
// random-number.service.ts
import { Injectable } from '@angular/core';

@Injectable()  // No providedIn
export class RandomNumberService {

  private randomValue: number;
```

```
  constructor() {
    this.randomValue = Math.floor(Math.random() * 1000);
  }

  getRandomNumber(): number {
    return this.randomValue;
  }

  generateNew(): void {
    this.randomValue = Math.floor(Math.random() * 1000);
  }
}
```

```
// component-a.component.ts
import { Component } from '@angular/core';
import { RandomNumberService } from './random-number.service';

@Component({
  selector: 'app-component-a',
  template: `
    <div>
      <h3>Component A</h3>
      <p>Random Number: {{ getNumber() }}</p>
      <button (click)="generate()">Generate New</button>
    </div>
  `,
  providers: [RandomNumberService]  // Component-level provider
})
export class ComponentAComponent {

  constructor(private randomService: RandomNumberService) { }

  getNumber(): number {
    return this.randomService.getRandomNumber();
  }

  generate(): void {
    this.randomService.generateNew();
  }
}
```

```
// component-b.component.ts
import { Component } from '@angular/core';
import { RandomNumberService } from './random-number.service';

@Component({
  selector: 'app-component-b',
  template: `
    <div>
      <h3>Component B</h3>
      <p>Random Number: {{ getNumber() }}</p>
      <button (click)="generate()">Generate New</button>
    </div>
  `,
  providers: [RandomNumberService]  // Separate instance
})
export class ComponentBComponent {

  constructor(private randomService: RandomNumberService) { }

  getNumber(): number {
    return this.randomService.getRandomNumber();
  }

  generate(): void {
    this.randomService.generateNew();
  }
}
```

Output:

Result: Component A and Component B each get their OWN instance of RandomNumberService. They have different random numbers, and changing one doesn't affect the other.

3.5 Provider Comparison

Provider Level	Configuration	Scope	Instance Sharing
Root	`providedIn: 'root'`	Entire app	Single instance shared everywhere
Module			

Provider Level	Configuration	Scope	Instance Sharing
	`providers: []` in @NgModule	Module and its components	One instance per module
Component	`providers: []` in @Component	Component and children	New instance per component

> **Important:**
>
> **Best Practice:** Use `providedIn: 'root'` for most services. Only use component or module-level providers when you specifically need separate instances.

4. Hierarchical Dependency Injection

4.1 The Injector Tree

Angular creates a hierarchy of injectors that mirrors your component tree. When a component requests a dependency, Angular searches up the injector tree until it finds a provider.

```
// Injector Hierarchy Example

AppModule Injector (Root)
    ├── ServiceA (provided in root)
    │
    └── AppComponent Injector
            ├── ServiceB (provided in AppComponent)
            │
            ├── ChildComponent Injector
            │       └── ServiceC (provided in ChildComponent)
            │
            └── SiblingComponent Injector
                    └── (inherits ServiceB from parent)
```

4.2 Dependency Resolution Process

When a component requests a dependency, Angular follows this search process:

1. Check the component's own injector
2. If not found, check the parent component's injector
3. Continue up the tree to the module injector
4. Finally check the root injector
5. If still not found, throw an error

```
// Resolution flow visualization

Component requests Service
    ↓
Check Component Injector → Found? → Inject
    ↓ Not found
Check Parent Injector → Found? → Inject
    ↓ Not found
Check Module Injector → Found? → Inject
    ↓ Not found
Check Root Injector → Found? → Inject
    ↓ Not found
ERROR: No provider found!
```

4.3 Practical Example: Hierarchical Services

Let's create a practical example showing how hierarchy affects service instances:

```
// theme.service.ts
import { Injectable } from '@angular/core';

@Injectable()
export class ThemeService {

  private currentTheme: string;

  constructor() {
    this.currentTheme = 'default';
    console.log('ThemeService created with theme:', this.currentTheme);
  }
```

```
  setTheme(theme: string): void {
    this.currentTheme = theme;
  }

  getTheme(): string {
    return this.currentTheme;
  }
}
```

```
// parent.component.ts
import { Component } from '@angular/core';
import { ThemeService } from './theme.service';

@Component({
  selector: 'app-parent',
  template: `
    <div>
      <h2>Parent Component</h2>
      <p>Theme: {{ getTheme() }}</p>
      <button (click)="changeTheme('blue')">Set Blue Theme</button>

      <app-child-one></app-child-one>
      <app-child-two></app-child-two>
    </div>
  `,
  providers: [ThemeService]  // New instance for parent and children
})
export class ParentComponent {

  constructor(private themeService: ThemeService) { }

  getTheme(): string {
    return this.themeService.getTheme();
  }

  changeTheme(theme: string): void {
    this.themeService.setTheme(theme);
  }
}
```

```
// child-one.component.ts
import { Component } from '@angular/core';
import { ThemeService } from './theme.service';
```

```
@Component({
  selector: 'app-child-one',
  template: `
    <div style="padding-left: 20px; border-left: 2px solid #ccc;">
      <h3>Child One</h3>
      <p>Theme: {{ getTheme() }}</p>
      <button (click)="changeTheme('red')">Set Red Theme</button>
    </div>
  `
  // No providers - inherits from parent
})
export class ChildOneComponent {

  constructor(private themeService: ThemeService) { }

  getTheme(): string {
    return this.themeService.getTheme();
  }

  changeTheme(theme: string): void {
    this.themeService.setTheme(theme);
  }
}
```

```
// child-two.component.ts
import { Component } from '@angular/core';
import { ThemeService } from './theme.service';

@Component({
  selector: 'app-child-two',
  template: `
    <div style="padding-left: 20px; border-left: 2px solid #ccc;">
      <h3>Child Two</h3>
      <p>Theme: {{ getTheme() }}</p>
      <button (click)="changeTheme('green')">Set Green Theme</button>
    </div>
  `,
  providers: [ThemeService]  // New instance, separate from parent
})
export class ChildTwoComponent {

  constructor(private themeService: ThemeService) { }

  getTheme(): string {
```

```
    return this.themeService.getTheme();
  }

  changeTheme(theme: string): void {
    this.themeService.setTheme(theme);
  }
}
```

Output:

Behavior:
• Parent and Child One share the SAME ThemeService instance
• When Child One sets theme to 'red', Parent also shows 'red'
• Child Two has its OWN ThemeService instance
• Child Two's theme changes don't affect Parent or Child One

Console output when app loads:
ThemeService created with theme: default (for Parent)
ThemeService created with theme: default (for Child Two)

4.4 Hierarchy Decision Tree

Use this decision tree to determine where to provide your service:

```
Should this service be shared across the entire app?
│
├─ YES → Use providedIn: 'root'
│         (Most common case)
│
└─ NO → Should each component have its own instance?
        │
        ├─ YES → Provide in component
        │         (e.g., form services, local state)
        │
        └─ NO → Should it be shared within a feature module?
                │
                └─ YES → Provide in module
                          (e.g., feature-specific services)
```

5. Constructor Injection Deep Dive

5.1 How Constructor Injection Works

The most common way to inject dependencies is through the constructor. Angular analyzes the constructor parameters and provides the required services.

```
// Multiple dependencies injection
import { Component } from '@angular/core';
import { UserService } from './user.service';
import { LoggerService } from './logger.service';
import { ConfigService } from './config.service';

@Component({
  selector: 'app-dashboard',
  template: `<div>Dashboard</div>`
})
export class DashboardComponent {

  // Inject multiple services
  constructor(
    private userService: UserService,
    private logger: LoggerService,
    private config: ConfigService
  ) {
    // Services are automatically injected and ready to use
    this.logger.log('Dashboard initialized');
    const appName = this.config.getAppName();
    const user = this.userService.getCurrentUser();
  }
}
```

Output:

Constructor Parameters:

- Each parameter declares a dependency
- `private` creates a class property automatically
- Angular provides instances based on types
- Services are available immediately in constructor

5.2 Access Modifiers in Constructor

The access modifier in the constructor parameter determines how the service can be accessed:

```
@Component({
  selector: 'app-example'
})
export class ExampleComponent {

  constructor(
    private privateService: PrivateService,    // Only within class
    public publicService: PublicService,       // Accessible in template
    protected protectedService: ProtectedService, // For inheritance
    readonly readonlyService: ReadonlyService  // Cannot be reassigned
  ) { }
}
```

Modifier	Class Access	Template Access	Common Usage
`private`	✓ Yes	✗ No	Most services (best practice)
`public`	✓ Yes	✓ Yes	When template needs direct access
`protected`	✓ Yes	✗ No	Inherited components
`readonly`	✓ Yes	✓ Yes	Prevent accidental reassignment

> **Tip:**
>
> **Best Practice:** Use `private` for most service injections. This encapsulates the service within the component and prevents template access. Create public methods in the component if template needs service functionality.

5.3 Shorthand Property Declaration

Constructor parameters with access modifiers automatically create class properties. These two are equivalent:

```
// Verbose approach (without shorthand)
export class VerboseComponent {

  userService: UserService;

  constructor(userService: UserService) {
    this.userService = userService;
  }
}
```

```
// Shorthand approach (recommended)
export class ShorthandComponent {

  // Property created and assigned automatically
  constructor(private userService: UserService) { }
}
```

> **Output:**
>
> **Both create the same result:** A class property named `userService` that holds the injected service instance. The shorthand is cleaner and preferred in Angular.

6. Services Injecting Services

6.1 Service Dependencies

Services can depend on other services, creating a dependency chain. This is common in real applications.

```
// writer.interface.ts
// Define a runtime token-friendly contract (interfaces can't be injected directly)
export interface IWriter { write(message: string): void }
```

```
// writers/console-writer.service.ts
import { Injectable } from '@angular/core';
import { IWriter } from './writer.interface';

@Injectable()
export class ConsoleWriter implements IWriter {
  write(message: string): void {
    console.log('[ConsoleWriter]', message);
  }
}

// writers/file-writer.service.ts
import { Injectable } from '@angular/core';
import { IWriter } from './writer.interface';

@Injectable()
export class FileWriter implements IWriter {
  write(message: string): void {
    // In a browser app you'd typically call an API to persist the log to a file/server.
    // This placeholder shows intent; implementation depends on platform (Node/Electron/
backend).
    console.log('[FileWriter] (simulated write to file):', message);
  }
}

// writers/db-writer.service.ts
import { Injectable } from '@angular/core';
import { IWriter } from './writer.interface';

@Injectable()
export class DbWriter implements IWriter {
  write(message: string): void {
    // Simulate persisting to DB (would usually use HttpClient in a real app)
    console.log('[DbWriter] (simulated write to DB):', message);
  }
}

// writer.token.ts
import { InjectionToken } from '@angular/core';
import { IWriter } from './writer.interface';
export const WRITER = new InjectionToken('WRITER');

// audit.service.ts - Consumer that depends on a writer implementation
import { Injectable, Inject } from '@angular/core';
import { WRITER } from './writer.token';
import { IWriter } from './writer.interface';
```

```
@Injectable({ providedIn: 'root' })
export class AuditService {
  constructor(@Inject(WRITER) private writer: IWriter) {
    this.writer.write('AuditService initialized');
  }

  record(event: string) {
    const payload = `${new Date().toISOString()} - ${event}`;
    this.writer.write(payload);
  }
}
```

```
// component using AuditService
import { Component } from '@angular/core';
import { AuditService } from './audit.service';

@Component({
  selector: 'app-data-manager',
  template: `
    <div>
      <h2>Data Manager</h2>
      <button (click)="addItem()">Add Item</button>
      <button (click)="clear()">Clear All</button>
      <p>Items: {{ getCount() }}</p>
    </div>
  `
})
export class DataManagerComponent {
  private items: any[] = [];

  constructor(private audit: AuditService) { }

  addItem(): void {
    const item = { id: Date.now(), name: 'Item' };
    this.items.push(item);
    this.audit.record('Added item: ' + JSON.stringify(item));
  }

  clear(): void {
    this.items = [];
    this.audit.record('Cleared items');
  }

  getCount(): number {
```

```
    return this.items.length;
  }
}
```

> **Output:**
>
> **Dependency Chain:**
> Component → AuditService → IWriter (ConsoleWriter / FileWriter / DbWriter)
>
> **How to swap implementations:**
> Provide a different class for the `WRITER` InjectionToken in `providers` (e.g., testing uses `ConsoleWriter`, production uses `DbWriter`) — the `AuditService` and components stay unchanged.
>
> **Sample Console Output (testing provider):**
> [ConsoleWriter] AuditService initialized
> [ConsoleWriter] 2026-01-28T10:30:20.000Z - Added item: {"id":...,"name":"Item"}
> [ConsoleWriter] 2026-01-28T10:30:25.000Z - Cleared items

6.2 Complex Service Dependencies

Let's build a more complex example with multiple service dependencies:

```
// config.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class ConfigService {

  private settings = {
    maxItems: 10,
    enableLogging: true,
    storageKey: 'app-data'
  };

  getMaxItems(): number {
    return this.settings.maxItems;
  }
```

```
  isLoggingEnabled(): boolean {
    return this.settings.enableLogging;
  }

  getStorageKey(): string {
    return this.settings.storageKey;
  }
}
```

```
// storage.service.ts - Uses ConfigService
import { Injectable } from '@angular/core';
import { ConfigService } from './config.service';

@Injectable({
  providedIn: 'root'
})
export class StorageService {

  constructor(private config: ConfigService) { }

  save(data: any): void {
    const key = this.config.getStorageKey();
    localStorage.setItem(key, JSON.stringify(data));
  }

  load(): any {
    const key = this.config.getStorageKey();
    const data = localStorage.getItem(key);
    return data ? JSON.parse(data) : null;
  }

  clear(): void {
    const key = this.config.getStorageKey();
    localStorage.removeItem(key);
  }
}
```

```
// task.service.ts - Uses ConfigService, LoggerService, StorageService
import { Injectable } from '@angular/core';
import { ConfigService } from './config.service';
import { LoggerService } from './logger.service';
import { StorageService } from './storage.service';
```

```
@Injectable({
  providedIn: 'root'
})
export class TaskService {

  private tasks: any[] = [];

  constructor(
    private config: ConfigService,
    private logger: LoggerService,
    private storage: StorageService
  ) {
    this.loadTasks();
  }

  private loadTasks(): void {
    this.tasks = this.storage.load() || [];
    if (this.config.isLoggingEnabled()) {
      this.logger.log(`Loaded ${this.tasks.length} tasks`);
    }
  }

  addTask(title: string): boolean {
    const maxItems = this.config.getMaxItems();

    if (this.tasks.length >= maxItems) {
      if (this.config.isLoggingEnabled()) {
        this.logger.log(`Cannot add task: limit reached (${maxItems})`);
      }
      return false;
    }

    const task = { id: Date.now(), title, completed: false };
    this.tasks.push(task);
    this.storage.save(this.tasks);

    if (this.config.isLoggingEnabled()) {
      this.logger.log(`Task added: ${title}`);
    }

    return true;
  }

  getTasks(): any[] {
    return this.tasks;
  }
```

```
  clearTasks(): void {
    this.tasks = [];
    this.storage.clear();

    if (this.config.isLoggingEnabled()) {
      this.logger.log('All tasks cleared');
    }
  }
}
```

Output:

Dependency Graph:
TaskService
├── ConfigService (configuration)
├── LoggerService (logging)
└── StorageService → ConfigService (storage with config)

Result: TaskService uses three services, and StorageService also depends on ConfigService. Angular manages all these dependencies automatically.

Important:

Important: The `@Injectable()` decorator is required on any service that has dependencies. Without it, Angular cannot inject dependencies into the service.

7. Practical Patterns and Best Practices

7.1 Service Communication Pattern

Services can be used to communicate between unrelated components (components that are not parent-child).

```
// message.service.ts - Communication service
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class MessageService {

  private message: string = '';

  setMessage(msg: string): void {
    this.message = msg;
  }

  getMessage(): string {
    return this.message;
  }

  clearMessage(): void {
    this.message = '';
  }
}
```

```
// sender.component.ts
import { Component } from '@angular/core';
import { MessageService } from './message.service';

@Component({
  selector: 'app-sender',
  template: `
    <div>
      <h3>Sender Component</h3>
      <input [(ngModel)]="inputMessage" placeholder="Enter message">
      <button (click)="send()">Send Message</button>
    </div>
  `
})
export class SenderComponent {

  inputMessage: string = '';

  constructor(private messageService: MessageService) { }

  send(): void {
```

```
    this.messageService.setMessage(this.inputMessage);
    this.inputMessage = '';
  }
}
```

```
// receiver.component.ts
import { Component } from '@angular/core';
import { MessageService } from './message.service';

@Component({
  selector: 'app-receiver',
  template: `
    <div>
      <h3>Receiver Component</h3>
      <p>Received: {{ getMessage() }}</p>
      <button (click)="clear()">Clear</button>
    </div>
  `
})
export class ReceiverComponent {

  constructor(private messageService: MessageService) { }

  getMessage(): string {
    return this.messageService.getMessage();
  }

  clear(): void {
    this.messageService.clearMessage();
  }
}
```

Output:

Result: SenderComponent and ReceiverComponent communicate through the shared MessageService. When Sender sends a message, Receiver can display it, even though they have no direct relationship.

7.2 Optional Dependencies

Sometimes a service might be optional. Angular provides the `@Optional()` decorator to handle this:

```
import { Component, Optional } from '@angular/core';
import { LoggerService } from './logger.service';

@Component({
  selector: 'app-optional-example',
  template: `<div>Check console</div>`
})
export class OptionalExampleComponent {

  constructor(@Optional() private logger: LoggerService) {
    // logger might be null if not provided
    if (this.logger) {
      this.logger.log('LoggerService is available');
    } else {
      console.log('LoggerService is not available');
    }
  }
}
```

Output:

Behavior:

- If LoggerService is provided → `logger` contains the service
- If LoggerService is NOT provided → `logger` is null
- No error is thrown if service is missing

7.3 Best Practices Summary

Practice	Recommendation	Reason
Service Scope	Use `providedIn: 'root'` by default	Tree-shakable, singleton, globally available
Constructor Modifier	Use `private` for injected services	Encapsulation and better design
@Injectable()	Always include on services	Enables dependency injection even if no current dependencies

Practice	Recommendation	Reason
Service Naming	Use `name.service.ts` pattern	Clear identification and consistency
Component-Level Providers	Use sparingly, only when needed	Creates new instances, breaks singleton pattern
Service Dependencies	Keep dependency chains reasonable	Avoid circular dependencies and complexity

8. Common DI Patterns

8.1 Factory Pattern with Services

```
// product-factory.service.ts
import { Injectable } from '@angular/core';

interface Product {
  id: number;
  name: string;
  category: string;
  price: number;
}

@Injectable({
  providedIn: 'root'
})
export class ProductFactoryService {

  private idCounter: number = 1;

  createProduct(name: string, category: string, price: number): Product {
    return {
      id: this.idCounter++,
      name: name,
      category: category,
      price: price
    };
  }

  createElectronicsProduct(name: string, price: number): Product {
```

```
    return this.createProduct(name, 'Electronics', price);
  }

  createClothingProduct(name: string, price: number): Product {
    return this.createProduct(name, 'Clothing', price);
  }
}
```

8.2 State Management Pattern

```
// app-state.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class AppStateService {

  private state = {
    isLoading: false,
    currentUser: null,
    theme: 'light',
    sidebarOpen: true
  };

  getState() {
    return { ...this.state };  // Return copy to prevent direct mutation
  }

  setLoading(loading: boolean): void {
    this.state.isLoading = loading;
  }

  isLoading(): boolean {
    return this.state.isLoading;
  }

  setCurrentUser(user: any): void {
    this.state.currentUser = user;
  }

  getCurrentUser(): any {
    return this.state.currentUser;
  }
```

```
  toggleSidebar(): void {
    this.state.sidebarOpen = !this.state.sidebarOpen;
  }

  isSidebarOpen(): boolean {
    return this.state.sidebarOpen;
  }
}
```

8.3 Validation Service Pattern

```
// validation.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class ValidationService {

  isEmail(value: string): boolean {
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return emailRegex.test(value);
  }

  isPhoneNumber(value: string): boolean {
    const phoneRegex = /^\d{10}$/;
    return phoneRegex.test(value);
  }

  isStrongPassword(value: string): boolean {
    // At least 8 characters, one uppercase, one lowercase, one number
    return value.length >= 8 &&
           /[A-Z]/.test(value) &&
           /[a-z]/.test(value) &&
           /[0-9]/.test(value);
  }

  isEmpty(value: string): boolean {
    return !value || value.trim().length === 0;
  }

  isInRange(value: number, min: number, max: number): boolean {
    return value >= min && value <= max;
```

```
  }

  getPasswordStrength(password: string): string {
    if (password.length < 6) return 'Weak';
    if (password.length < 10) return 'Medium';
    if (this.isStrongPassword(password)) return 'Strong';
    return 'Medium';
  }
}
```

9. Troubleshooting DI Issues

9.1 Common Errors and Solutions

Error 1: NullInjectorError

```
ERROR NullInjectorError: No provider for MyService!
```

Cause: Service is not provided anywhere.

Solution:

```
// Add @Injectable with providedIn
@Injectable({
  providedIn: 'root'  // ← Add this
})
export class MyService { }
```

Error 2: Circular Dependency

```
ERROR: Cannot instantiate cyclic dependency!
```

Cause: ServiceA depends on ServiceB, and ServiceB depends on ServiceA.

Solution: Refactor to remove circular dependency by introducing a third service or restructuring dependencies.

Error 3: Wrong Instance

Problem: Components have different service instances when you expected them to share.

Cause: Service provided at component level instead of root.

Solution:

```
// Remove providers from component
@Component({
  selector: 'app-my-component',
  // providers: [MyService]  ← Remove this
})

// Keep only in service
@Injectable({
  providedIn: 'root'  // ← This ensures singleton
})
```

9.2 Debugging Tips

```
// Add logging to track service creation
@Injectable({
  providedIn: 'root'
})
export class DebugService {

  constructor() {
    console.log('DebugService created at:', new Date().toISOString());
    console.trace('Created from:');  // Shows call stack
  }
}
```

> **Tip:**
>
> **Debugging Checklist:**
> ✓ Is `@Injectable()` present on the service?
> ✓ Is `providedIn: 'root'` set for singleton services?

✓ Are you importing from the correct path?
✓ Is the service listed in any component's providers array when it shouldn't be?
✓ Check for typos in service names and paths

10. Summary

Important:

Key Takeaways:

1. What is Dependency Injection?
• Design pattern where classes receive dependencies from external sources
• Angular's DI system automatically provides required dependencies
• Implements Inversion of Control principle

2. How DI Works:
• Injector creates and manages service instances
• Provider tells injector how to create services
• Dependencies declared in constructor parameters
• Angular handles entire injection process

3. Providers:
• Root-level: `providedIn: 'root'` (recommended)
• Module-level: `providers` array in @NgModule
• Component-level: `providers` array in @Component
• Different levels create different scopes

4. Hierarchical DI:
• Angular creates an injector tree
• Child components inherit parent providers
• Resolution searches up the tree
• Component providers create new instances

5. Best Practices:
• Use `providedIn: 'root'` by default

- Use `private` for constructor parameters
- Always include `@Injectable()` on services
- Avoid circular dependencies
- Keep dependency chains manageable

11. Practice Exercise

Note:

Exercise: Build a Shopping Cart System with DI

Create a shopping cart system that demonstrates hierarchical dependency injection:

1. Create Services:
- `ProductService` - Manages product catalog (root-level)
- `CartService` - Manages cart items, depends on ProductService (root-level)
- `PriceCalculatorService` - Calculates totals, tax, etc. (root-level)
- `NotificationService` - Shows messages (component-level for different instances)

2. ProductService Methods:
- `getProducts()` - Return list of products
- `getProductById(id)` - Find specific product
- Initialize with 5 sample products

3. CartService Methods (depends on ProductService):
- `addToCart(productId)` - Add product to cart
- `removeFromCart(productId)` - Remove from cart
- `getCartItems()` - Return cart items
- `getCartCount()` - Return number of items
- `clearCart()` - Empty the cart

4. PriceCalculatorService Methods:
- `calculateSubtotal(items)` - Sum of item prices
- `calculateTax(subtotal)` - 10% tax

- `calculateTotal(items)` - Subtotal + tax

5. Components:
- `ProductListComponent` - Display products, inject ProductService and CartService
- `CartComponent` - Show cart, inject CartService and PriceCalculatorService
- Each component should have its own NotificationService instance

6. Test:
- Add products from ProductListComponent
- Verify CartComponent shows same items (shared CartService)
- Notifications should be independent per component
- Calculate and display cart total with tax

Next: 6.3 Service Scoping - Understanding singleton objects, tree-shakability, and advanced provider configurations